cJSON 函数说明文档

库版本：1.7.19

源文件：cJSON.h / cJSON.c（纯 C89，直接加入工程即可）

本文档基于对两个源文件的完整阅读整理。

目录

1. 核心数据结构
2. 类型常量
3. 解析函数
4. 打印（序列化）函数
5. 销毁函数
6. 类型判断函数
7. 取值函数
8. 创建函数
9. 添加函数
10. 查找函数
11. 删除与分离函数
12. 修改与替换函数
13. 复制与比较函数
14. 工具函数
15. 内存释放规则速查

1. 核心数据结构

```
typedef struct cJSON {
  struct cJSON *next;   /* 下一个兄弟节点 */
  struct cJSON *prev;   /* 上一个兄弟节点 */
  struct cJSON *child;  /* 数组/对象的第一个子节点 */
  int  type;        /* 节点类型（见类型常量） */
  char *valuestring;    /* 字符串/Raw 的值 */
  int  valueint;      /* 整数值（已弃用，勿直接写） */
  double valuedouble;   /* 数字值 */
  char *string;       /* 对象子项的键名 */
} cJSON;
```

- 数组和对象的元素通过 child 指向的双向链表组织
- 对象的每个子节点用 string 字段保存键名
- 附加标志位（与 type 按位或存在）：

标志	值	含义
cJSON_IsReference	256	引用节点，Delete 时不释放其 child/valuestring
cJSON_StringIsConst	512	键名为常量字符串，Delete 时不释放 string

2. 类型常量

宏	值	JSON 形态
cJSON_Invalid	0	无效
cJSON_False	1	false
cJSON_True	2	true
cJSON_NULL	4	null
cJSON_Number	8	123 / 3.14
cJSON_String	16	"abc"
cJSON_Array	32	[1,2]
cJSON_Object	64	{"k":1}
cJSON_Raw	128	原始 JSON 片段

判断真实类型时应使用 type & 0xFF（屏蔽标志位），推荐直接使用 cJSON_Is* 系列函数。

安全限制宏（编译期可重定义）：

宏	默认值	作用
CJSON_NESTING_LIMIT	1000	最大嵌套深度，防栈溢出
CJSON_CIRCULAR_LIMIT	10000	Duplicate 防循环引用

3. 解析函数

cJSON_Parse

```
cJSON *cJSON_Parse(const char *value);
```

- 功能：解析以 \0 结尾的 JSON 字符串，返回 cJSON 树
- 返回：成功返回根节点；失败返回 NULL
- 注意：调用者必须用 cJSON_Delete 释放返回值

cJSON_ParseWithLength

```
cJSON *cJSON_ParseWithLength(const char *value, size_t buffer_length);
```

- 功能：按显式长度解析，适用于缓冲区非 null 结尾场景
- 参数：value 数据指针；buffer_length 数据长度

cJSON_ParseWithOpts

```
cJSON *cJSON_ParseWithOpts(const char *value, const char **return_parse_end,
                           cJSON_bool require_null_terminated);
```

- 功能：带选项解析
- 参数：
- return_parse_end：返回解析停止位置；失败时指向错误位置
- require_null_terminated：非 0 时要求 JSON 之后必须是 \0（即不允许尾部垃圾数据）

cJSON_ParseWithLengthOpts

```
cJSON *cJSON_ParseWithLengthOpts(const char *value, size_t buffer_length,
                                 const char **return_parse_end,
                                 cJSON_bool require_null_terminated);
```

- 功能：显式长度 + 完整选项，其余解析函数的底层实现

cJSON_GetErrorPtr

```
const char *cJSON_GetErrorPtr(void);
```

- 功能：返回解析失败位置的全局指针（需向前回看几个字符定位问题）
- 仅在 cJSON_Parse 返回 NULL 时有意义

4. 打印（序列化）函数

函数	说明
char *cJSON_Print(const cJSON *item)	格式化输出（带缩进换行），返回 malloc 字符串，需 free
char *cJSON_PrintUnformatted(const cJSON *item)	紧凑输出（无空白），需 free
char *cJSON_PrintBuffered(const cJSON *item, int prebuffer, cJSON_bool fmt)	预分配缓冲策略输出；prebuffer 是预估大小，fmt=1 格式化
cJSON_bool cJSON_PrintPreallocated(cJSON *item, char *buffer, const int length, const cJSON_bool format)	写入调用者提供的 buffer，成功返回 1；建议多预留 5 字节
void cJSON_Minify(char *json)	原地压缩 JSON（去空白和注释）；入参必须是可写缓冲区

示例：

```
char *s = cJSON_PrintUnformatted(root);
puts(s);
free(s);  /* 不能用 cJSON_Delete 释放字符串！ */
```

5. 销毁函数

```
void cJSON_Delete(cJSON *item);
```

- 递归释放整棵树（沿 next 链和 child 链）
- 标记 cJSON_IsReference 的节点不释放其引用目标
- 传入 NULL 安全

6. 类型判断函数

全部接受 NULL（返回 false），返回 cJSON_bool（1/0）：

函数	判断
cJSON_IsInvalid(item)	无效节点
cJSON_IsFalse(item)	false
cJSON_IsTrue(item)	true
cJSON_IsBool(item)	true 或 false

函数	判断
cJSON_IsNull(item)	null
cJSON_IsNumber(item)	数字
cJSON_IsString(item)	字符串
cJSON_IsArray(item)	数组
cJSON_IsObject(item)	对象
cJSON_IsRaw(item)	Raw 片段

7. 取值函数

```
char  *cJSON_GetStringValue(const cJSON * const item);  /* 非字符串返回 NULL */
double cJSON_GetNumberValue(const cJSON * const item);  /* 非数字返回 NaN */
```

等价于先判类型再读 item->valuestring / item->valuedouble，更安全。

8. 创建函数

基础类型

函数	说明
cJSON_CreateNull()	null 节点
cJSON_CreateTrue() / cJSON_CreateFalse()	布尔节点
cJSON_CreateBool(cJSON_bool boolean)	按参数创建布尔
cJSON_CreateNumber(double num)	数字节点（valueint 自动饱和到 INT_MAX/INT_MIN）
cJSON_CreateString(const char *string)	字符串节点（内部拷贝）
cJSON_CreateRaw(const char *raw)	Raw 节点，打印时原样输出，不转义
cJSON_CreateArray()	空数组
cJSON_CreateObject()	空对象

引用创建（不拥有目标，Delete 时跳过）

函数	说明
cJSON_CreateStringReference(const char *string)	引用外部字符串，不拷贝
cJSON_CreateObjectReference(const cJSON *child)	引用外部对象子树
cJSON_CreateArrayReference(const cJSON *child)	引用外部数组子树

从 C 数组批量创建 JSON 数组

函数	说明
cJSON_CreateIntArray(const int *numbers, int count)	int 数组 → JSON 数组
cJSON_CreateFloatArray(const float *numbers, int count)	float 数组 → JSON 数组
cJSON_CreateDoubleArray(const double *numbers, int count)	double 数组 → JSON 数组
cJSON_CreateStringArray(const char *const *strings, int count)	字符串数组 → JSON 数组

9. 添加函数

底层添加

```
cJSON_bool cJSON_AddItemToArray(cJSON *array, cJSON *item);
cJSON_bool cJSON_AddItemToObject(cJSON *object, const char *string, cJSON *item);
cJSON_bool cJSON_AddItemToObjectCS(cJSON *object, const char *string, cJSON *item);
```

- 添加后 item 所有权转移给父节点，不得再单独 Delete
- AddItemToObjectCS：键名按常量处理（不拷贝），要求键的生命周期长于树

引用添加（不转移所有权）

```
cJSON_bool cJSON_AddItemReferenceToArray(cJSON *array, cJSON *item);
cJSON_bool cJSON_AddItemReferenceToObject(cJSON *object, const char *string, cJSON *item);
```

- 同一节点可挂到多处；删除父节点不影响原节点
- 注意：必须先删引用方，再删被引用方

对象便捷添加（创建+添加一步完成，返回新节点）

函数	说明
cJSON_AddNullToObject(obj, name)	添加 null
cJSON_AddTrueToObject(obj, name) / cJSON_AddFalseToObject(obj, name)	添加布尔常量
cJSON_AddBoolToObject(obj, name, boolean)	添加布尔
cJSON_AddNumberToObject(obj, name, number)	添加数字
cJSON_AddStringToObject(obj, name, string)	添加字符串
cJSON_AddRawToObject(obj, name, raw)	添加 Raw 片段
cJSON_AddObjectToObject(obj, name)	创建并添加子对象，返回子对象指针
cJSON_AddArrayToObject(obj, name)	创建并添加子数组，返回子数组指针

10. 查找函数

函数	说明
int cJSON_GetArraySize(const cJSON *array)	数组/对象元素个数（链表遍历）
cJSON *cJSON_GetArrayItem(const cJSON *array, int index)	按索引取元素，失败返回 NULL
cJSON *cJSON_GetObjectItem(obj, string)	按键查找，不区分大小写
cJSON *cJSON_GetObjectItemCaseSensitive(obj, string)	按键查找，区分大小写（推荐）
cJSON_bool cJSON_HasObjectItem(object, string)	键是否存在（不区分大小写）

遍历宏：

```
cJSON *elem = NULL;
cJSON_ArrayForEach(elem, array_or_object) {
  /* elem 依次为每个子节点；对象场景下 elem-&gt;string 为键名 */
}
```

11. 删除与分离函数

Delete 系列：从父节点移除并立即释放内存。

函数	说明
cJSON_DeleteItemFromArray(array, which)	按索引删除数组元素
cJSON_DeleteItemFromObject(object, string)	按键删除（不区分大小写）
cJSON_DeleteItemFromObjectCaseSensitive(object, string)	按键删除（区分大小写）

Detach 系列：从父节点移除但不释放，返回被摘下的节点指针（调用者负责后续 Delete）。

函数	说明
cJSON_DetachItemViaPointer(parent, item)	按指针分离（底层实现）
cJSON_DetachItemFromArray(array, which)	按索引分离数组元素
cJSON_DetachItemFromObject(object, string)	按键分离（不区分大小写）
cJSON_DetachItemFromObjectCaseSensitive(object, string)	按键分离（区分大小写）

12. 修改与替换函数

改值（宏/辅助函数）

接口	说明
cJSON_SetIntValue(object, number)	宏：同时写 valueint 和 valuedouble
cJSON_SetNumberValue(object, number)	设置数字值（推荐）
cJSON_SetBoolValue(object, boolValue)	宏：仅对布尔节点有效，改 true/false
cJSON_SetValuestring(object, valuestring)	改字符串值，仅对 cJSON_String 节点有效；新串更长时会重新分配

替换（旧节点被 Delete，新节点接管其位置）

函数	说明
cJSON_ReplaceItemViaPointer(parent, item, replacement)	按指针替换
cJSON_ReplaceItemInArray(array, which, newitem)	按索引替换数组元素
cJSON_ReplaceItemInObject(object, string, newitem)	按键替换（不区分大小写，键名重新拷贝）
cJSON_ReplaceItemInObjectCaseSensitive(object, string, newitem)	按键替换（区分大小写）

插入

```
cJSON_bool cJSON_InsertItemInArray(cJSON *array, int which, cJSON *newitem);
```

- 在索引 which 处插入，原有元素右移；which 越界时追加到末尾

13. 复制与比较函数

```
cJSON *cJSON_Duplicate(const cJSON *item, cJSON_bool recurse);
```

- 深拷贝节点；recurse 非 0 时递归复制所有子节点
- 返回新树，需独立 cJSON_Delete

```
cJSON_bool cJSON_Compare(const cJSON * const a, const cJSON * const b,
                         const cJSON_bool case_sensitive);
```

- 递归比较两棵树是否相等；case_sensitive 决定对象键名是否区分大小写
- 数字用浮点容差比较

14. 工具函数

函数	说明
const char *cJSON_Version(void)	返回版本字符串，如 "1.7.19"
void cJSON_InitHooks(cJSON_Hooks *hooks)	自定义 malloc/free（嵌入式场景）；传 NULL 恢复默认
void *cJSON_malloc(size_t size)	用当前 hooks 分配内存
void cJSON_free(void *object)	用当前 hooks 释放内存

自定义分配器示例：

```
cJSON_Hooks hooks = { my_malloc, my_free };
cJSON_InitHooks(&amp;hooks);  /* 之后 cJSON_Print 返回的字符串也应配对释放 */
```

15. 内存释放规则速查

来源	释放方式
cJSON_Parse* 返回的树	cJSON_Delete
cJSON_Create* 创建后未挂载的节点	cJSON_Delete
已 AddItem 挂载的节点	随父节点 Delete 自动释放，禁止单独 Delete
cJSON_Print / PrintUnformatted / PrintBuffered 返回的字符串	free（或配对的 cJSON_free）
cJSON_Duplicate 返回的副本	cJSON_Delete
cJSON_Detach* 摘下的节点	cJSON_Delete
引用节点（IsReference）	只释放引用壳，不动目标；先删引用方再删目标